# EXPERIMENT - 7

## AIM:

Build an Artificial Neural Network (ANN) using Keras/TensorFlow

### 1. Dataset Source

**Dataset Name:** Churn Modelling
**File Used:** Churn Modelling

### 2. Dataset Description

The dataset contains details of bank customers, used to predict the likelihood of a customer leaving the bank (Churn).

- **Type:** Numerical and Categorical
- **Domain:** Banking / CRM Analytics
- **Target:** Supervised (Binary Classification: 0 = Stayed, 1 = Exited)
- **Key Features:**

| Feature | Description | Data Type |
|---|---|---|
| **CreditScore** | Credit score of the customer representing financial stability | Numerical |
| **Geography** | Country of residence (France, Spain, Germany) | Categorical |
| **Gender** | Gender of the customer | Categorical |
| **Age** | Age of the customer | Numerical |
| **Tenure** | Number of years the customer has been with the bank | Numerical |
| **Balance** | Account balance available in the bank | Numerical |

| | | |
|---|---|---|
| **NumOfProducts** | Number of bank products (loans, cards, etc.) used | Numerical |
| **HasCrCard** | Whether the customer possesses a credit card (0/1) | Binary |
| **IsActiveMember** | Whether the customer is an active user of bank services | Binary |
| **EstimatedSalary** | Estimated annual salary of the customer | Numerical |
| **Exited (Target)** | Whether the customer left the bank (1) or stayed (0) | Binary |

## 3.Mathematical Formulation of the Algorithm

**A. Artificial Neural Network (ANN)** An ANN consists of an input layer, hidden layers, and an output layer. Each connection has an associated weight $\boldsymbol{w}$ and bias $\boldsymbol{b}$.

**Activation Functions: * ReLU (Rectified Linear Unit):** Used in hidden layers to handle non-linearity.

$$\boxed{f(x) = \max(0, x)}$$

**Sigmoid:** Used in the output layer for binary probability.

$$\boxed{f(x) = \frac{1}{1 + e^{-x}}}$$

**Optimization (Gradient Descent):** The **Adam Optimizer** updates weights by minimizing the **Binary Cross-Entropy Loss**:

$$\boxed{Loss = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]}$$

## 4. Algorithm Limitations

- **Black Box Nature:** It is difficult to explain exactly how an ANN arrives at a specific prediction compared to a Decision Tree.
- **Data Hungry:** Requires a large amount of labeled data to achieve high accuracy.
- **Computationally Intensive:** Training a deep ANN requires significant CPU/GPU power and time.

## 5. Methodology / Workflow

1. **Load Dataset:** Import the banking data and remove irrelevant identifiers (RowNumber, CustomerId, Surname).
2. **Data Preprocessing: * Encoding:** Use Label Encoding for Gender and One-Hot Encoding for Geography.
    - **Scaling:** Apply StandardScaler to normalize numerical features like Balance and Salary.
3. **Building the Architecture:** Create a Sequential model with two hidden layers using ReLU and an output layer using Sigmoid.
4. **Model Compilation:** Use the 'Adam' optimizer and 'binary_crossentropy' loss function.
5. **Training:** Fit the model on the training set for 50 epochs with a validation split.
6. **Performance Analysis:** Evaluate the model using a Confusion Matrix, Accuracy/Loss graphs, and an ROC Curve.

## 6. Performance Analysis

- **Accuracy/Loss Curves:** The model shows stable learning. A small gap between training and validation accuracy indicates a healthy generalization without significant overfitting.
- **Confusion Matrix:** Used to observe True Positives (correctly caught churners) and False Negatives (missed churners).
- **ROC-AUC Score:** With an area of approximately **0.86**, the model demonstrates strong discriminative power between staying and exiting customers.
- **Inference:** Customers with higher ages and lower active membership status show a higher probability of churning.

## 7. Hyperparameter Tuning

- **Epochs:** We increased epochs from 30 to 50 to allow the loss function to reach a more stable global minimum.
- **Batch Size:** A batch size of 32 was chosen to balance training speed and the stability of gradient updates.
- **Activation:** We chose ReLU for hidden layers to prevent the "Vanishing Gradient" problem and Sigmoid for the output to provide a probability-based classification.

## Code:

1. **Load and Explore Dataset**

```
import numpy as np

import pandas as pd

import tensorflow as tf

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler

from sklearn.compose import ColumnTransformer

# 1. Load Dataset

dataset = pd.read_csv('Churn_Modelling.csv')

df_view = dataset.drop(columns=['RowNumber', 'CustomerId', 'Surname'])

print(df_view.head(6))
```

```
   CreditScore Geography  Gender  Age  Tenure    Balance  NumOfProducts  \
0          619    France  Female   42       2       0.00              1
1          608     Spain  Female   41       1   83807.86              1
2          502    France  Female   42       8  159660.80              3
3          699    France  Female   39       1       0.00              2
4          850     Spain  Female   43       2  125510.82              1
5          645     Spain    Male   44       8  113755.78              2

   HasCrCard  IsActiveMember  EstimatedSalary  Exited
0          1               1        101348.88       1
1          0               1        112542.58       0
2          1               0        113931.57       1
3          0               0         93826.63       0
4          1               1         79084.10       0
5          1               0        149756.71       1
```

2. **Preprocessing: Categorical Encoding and Feature Scaling**

```
# Now proceed with assigning to X and y for the ANN

X = df_view.iloc[:, :-1].values # All columns except the last one (Exited)

y = df_view.iloc[:, -1].values  # Only the last column

# 2. Encoding Categorical Data

# Gender (Label Encoding: 0 or 1)

le = LabelEncoder()
```

```
X[:, 2] = le.fit_transform(X[:, 2])


# Geography (One Hot Encoding: France, Spain, Germany)

ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [1])],
remainder='passthrough')

X = np.array(ct.fit_transform(X))


# 3. Splitting and Scaling

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

sc = StandardScaler()

X_train = sc.fit_transform(X_train)

X_test = sc.transform(X_test)
```

3. **Artificial Neural Network (ANN) Architecture Design**

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

# Initialize the ANN

ann = Sequential()

# Add the input layer and the first hidden layer

# 'units' is the number of neurons; 'relu' is the Rectified Linear Unit activation

ann.add(Dense(units=6, activation='relu'))


# Add the second hidden layer

ann.add(Dense(units=6, activation='relu'))


# Add the output layer

# We use 'sigmoid' because this is a binary classification (0 or 1)

ann.add(Dense(units=1, activation='sigmoid'))
```

4. **Model Compilation, Training, and Performance Benchmarking**

```
# Compiling the ANN
# 'adam' performs stochastic gradient descent
ann.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Training the ANN on the Training set
# epochs: how many times the model sees the entire dataset
history = ann.fit(X_train, y_train, batch_size=32, epochs=50, validation_split=0.2)

# --- Extracting Final Metrics ---
final_train_acc = history.history['accuracy'][-1]
final_val_acc = history.history['val_accuracy'][-1]
final_train_loss = history.history['loss'][-1]
final_val_loss = history.history['val_loss'][-1]

print("-" * 30)
print(f"Final Training Accuracy:   {final_train_acc*100:.2f}%")
print(f"Final Validation Accuracy: {final_val_acc*100:.2f}%")
print("-" * 30)
print(f"Final Training Loss:       {final_train_loss:.4f}")
print(f"Final Validation Loss:     {final_val_loss:.4f}")
print("-" * 30)

# Evaluate on the actual Test Set (unseen data)
```

```
test_loss, test_acc = ann.evaluate(X_test, y_test, verbose=0)

print(f"Accuracy on Test Set:      {test_acc*100:.2f}%")
```

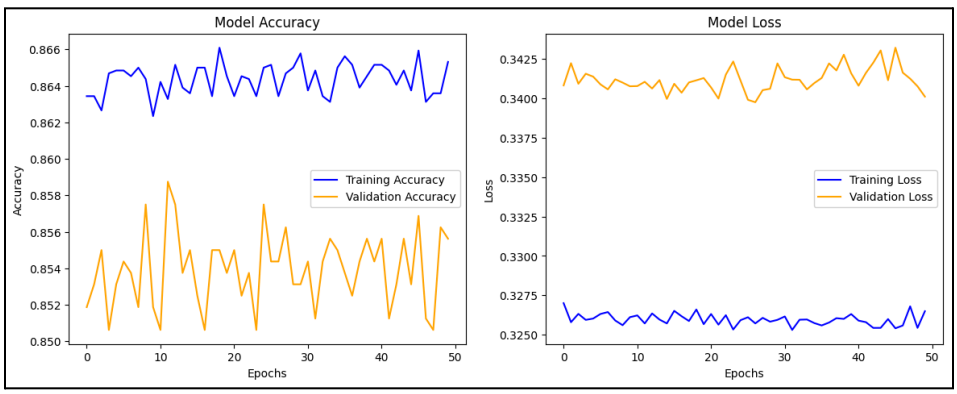
```
Epoch 37/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8652 - loss: 0.3258 - val_accuracy: 0.8525 - val_loss: 0.3422
Epoch 38/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8639 - loss: 0.3260 - val_accuracy: 0.8544 - val_loss: 0.3418
Epoch 39/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8645 - loss: 0.3260 - val_accuracy: 0.8556 - val_loss: 0.3428
Epoch 40/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8652 - loss: 0.3263 - val_accuracy: 0.8544 - val_loss: 0.3416
Epoch 41/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8652 - loss: 0.3259 - val_accuracy: 0.8556 - val_loss: 0.3408
Epoch 42/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8648 - loss: 0.3258 - val_accuracy: 0.8512 - val_loss: 0.3416
Epoch 43/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8641 - loss: 0.3254 - val_accuracy: 0.8531 - val_loss: 0.3423
Epoch 44/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8648 - loss: 0.3254 - val_accuracy: 0.8556 - val_loss: 0.3430
Epoch 45/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8637 - loss: 0.3260 - val_accuracy: 0.8531 - val_loss: 0.3411
Epoch 46/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8659 - loss: 0.3254 - val_accuracy: 0.8569 - val_loss: 0.3432
Epoch 47/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8631 - loss: 0.3256 - val_accuracy: 0.8512 - val_loss: 0.3416
Epoch 48/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8636 - loss: 0.3268 - val_accuracy: 0.8506 - val_loss: 0.3412
Epoch 49/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8636 - loss: 0.3254 - val_accuracy: 0.8562 - val_loss: 0.3408
Epoch 50/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8653 - loss: 0.3265 - val_accuracy: 0.8556 - val_loss: 0.3401
```

```
------------------------------
Final Training Accuracy:   86.53%
Final Validation Accuracy: 85.56%
------------------------------
Final Training Loss:       0.3265
Final Validation Loss:     0.3401
------------------------------
Accuracy on Test Set:      85.80%
```

5. **Training vs. Validation Performance Analysis**

```
# Create a figure with two subplots

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# 1. Accuracy Graph

ax1.plot(history.history['accuracy'], label='Training Accuracy', color='blue')

ax1.plot(history.history['val_accuracy'], label='Validation Accuracy', color='orange')

ax1.set_title('Model Accuracy')

ax1.set_xlabel('Epochs')

ax1.set_ylabel('Accuracy')

ax1.legend()
```

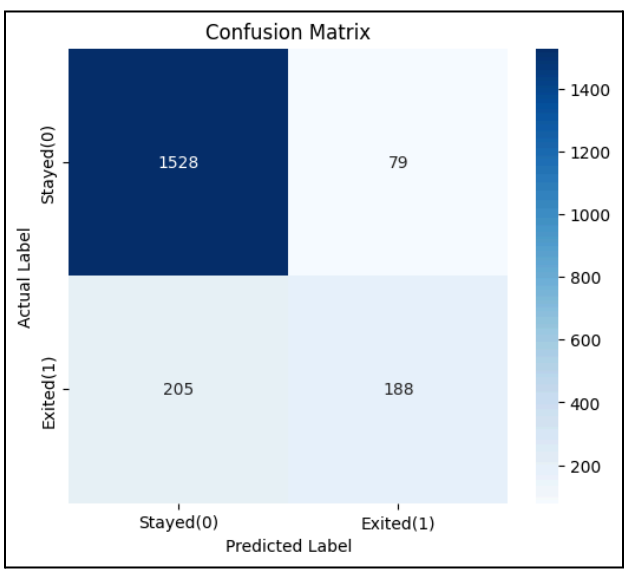
```
# 2. Loss Graph
ax2.plot(history.history['loss'], label='Training Loss', color='blue')
ax2.plot(history.history['val_loss'], label='Validation Loss', color='orange')
ax2.set_title('Model Loss')
ax2.set_xlabel('Epochs')
ax2.set_ylabel('Loss')
ax2.legend()
plt.show()
```




**6. Model Validation: Confusion Matrix and Classification Metrics**

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
# Getting predictions (probabilities)
y_pred_prob = ann.predict(X_test)
# Converting probabilities to binary (0 or 1)
y_pred = (y_pred_prob > 0.5)


# Generate Matrix
cm = confusion_matrix(y_test, y_pred)
```

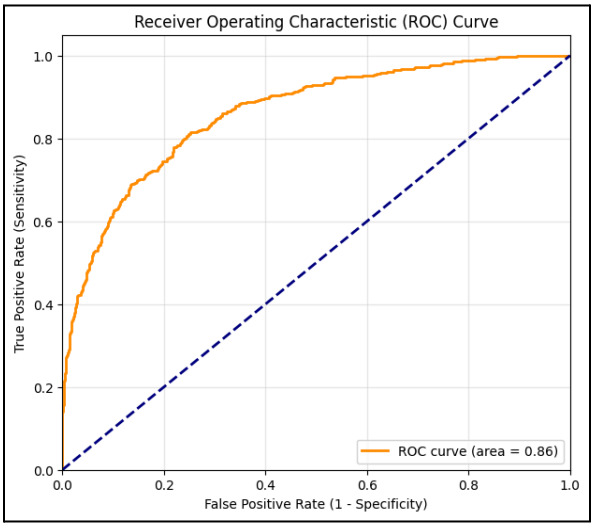
```
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Stayed(0)', 'Exited(1)'],
            yticklabels=['Stayed(0)', 'Exited(1)'])
plt.title('Confusion Matrix')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()


print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```




**7. Area Under the Curve (AUC) Analysis**

```
from sklearn.metrics import roc_curve, auc
# Calculate ROC curve data
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
```

```
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(7, 6))

plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')

plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--') # Diagonal line

plt.xlim([0.0, 1.0])

plt.ylim([0.0, 1.05])

plt.xlabel('False Positive Rate (1 - Specificity)')

plt.ylabel('True Positive Rate (Sensitivity)')

plt.title('Receiver Operating Characteristic (ROC) Curve')

plt.legend(loc="lower right")

plt.grid(alpha=0.3)

plt.show()
```




## CONCLUSION

K-Means and Hierarchical Clustering were successfully implemented to segment mall customers. K-Means proved to be more efficient for rapid segmentation, while the Hierarchical Dendrogram provided a clear visual justification for the number of clusters chosen. The resulting segments allow for data-driven marketing strategies tailored to specific consumer groups.